试卷： 高级语言程序设计第二次机考模拟题

普通编程题(100分)

1. (20分) 用递归方法编程计算Fibonacci数列的前N项。其中fib(0)=0,fib(1)=1,fib(n)=fib(n-1)+fib(n-2)。

**输入格式要求："%d" 提示信息："Input n:"
**输出格式要求："Fib(%d)=%d\n"
程序运行示例如下：

```
Input n:10
Fib(1)=1
Fib(2)=1
Fib(3)=2
Fib(4)=3
Fib(5)=5
Fib(6)=8
Fib(7)=13
Fib(8)=21
Fib(9)=34
Fib(10)=55
```

```c
#include <stdio.h>
long Fib(int a);
int main()
{
    int n, i, x;
    printf("Input n:");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        x = Fib(i);     /* 调用递归函数Fib()计算Fibonacci数列的第n项 */
        printf("Fib(%d)=%d\n", i, x);
    }
    return 0;
}
/* 函数功能：用递归法计算Fibonacci数列中的第n项的值 */
long Fib(int n)
{
    long f;
    if (n == 0)    f = 0;                /* 基线情况 */
    else if (n == 1)    f = 1;          /* 基线情况 */
    else  f = Fib(n - 1) + Fib(n - 2); /* 一般情况 */
    return f;
}
```

2.（20分）字符串逆序

用字符数组作函数参数编程，利用一个数组实现字符串（允许输入带空格的字符串）的逆序存放。要求如下：
（1）在主函数中从键盘输入字符串，字符串的最大长度为80个字符。
调用Inverse()函数将字符串逆序存放，然后在主函数中输出逆序后的字符串。
（2）在子函数Inverse()中实现字符串的逆序存放。函数原型为：
void Inverse(char str[]);

程序运行结果示例1：
```
Input a string:
abcde↙
Inversed results:
edcba
```

程序运行结果示例2：
```
Input a string:
hello↙
Inversed results:
olleh
```

输入格式： 用gets()输入字符串
输出格式：
输入提示信息："Input a string:\n"
输出提示信息："Inversed results:\n"
用puts()输出字符串

```c
#include <stdio.h>
#include <string.h>
#define N 80
void Inverse(char str[]);
int main()
{
    char a[N];
    printf("Input a string:\n");
    gets(a);
    Inverse(a);
    printf("Inversed results:\n");
    puts(a);
    return 0;
}

void Inverse(char str[])
{
    int len, i, j;
    char temp;
    len = strlen(str);
    for (i = 0, j = len - 1; i < j; i++, j--)
    {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
    }
}
```

3. (20分) 编写函数void fun(int x, int *pp, int *n)，它的功能是：求出x的偶数因子，并按从小到大的顺序存放在pp所指的数组中，这些因子的个数通过形参n返回。如，若x中的值为24，则有6个数符合要求，它们是2、4、6、8、12、24。主函数中输入一个小于等于1000的正整数，调用fun函数之后按顺序输出该正整数的所有偶数因子。

输入输出形式如：

```
Input number:512
 2 4 8 16 32 64 128 256 512
```

最后以回车结束。

输入格式："%d"

输出格式：" %d"

```c
#include<stdio.h>

void fun(int x, int pp[], int *n)
{
    int i, j = 0;
    for (i = 2; i <= x; i = i + 2) //1
    {
        if (x % i == 0) //1
        {
            pp[j++] = i; //2
        }
    }
    *n = j; //1
}

void main()
{
    int x, aa[1000], n, i;
    printf("Input number:");  //1
    scanf("%d", &x);
    fun(x, aa, &n); //1
    for (i = 0; i < n; i++) //1
        printf(" %d", aa[i]); //1
    printf("\n"); //1
}
```

4.（20分）用链表实现对学生成绩进行存储、插入及显示功能，学生包括ID和score属性。

结构体类型定义如下：

```
struct grade {
        int score;
        char ID[6];
        struct grade *next;
};
```

（1）写函数创建单向链表并存储学生成绩信息，输入ID和score按逗号隔开，先输入score后输入ID，当score<0时链表创建完成，并输出创建后的链表信息。

（2）写函数实现在第n个结点后面插入一个新结点，并输出插入新结点后的链表信息。

（3）写函数释放链表所占内存空间。

（在主函数依次调用各函数即可）

程序运行示例：

输入学生成绩和ID：

```
23,01
45,02
67,04
-1
```

创建的链表：

```
23,01
45,02
67,04
```

请输入要插入结点的成绩和ID：

```
78,02
```

请输入要插入结点的位置,在这个位置后面插入新结点：

```
3
```

插入后的链表：

```
23,01
45,02
67,04
78,02
```

程序运行示例2：

输入学生成绩和ID：

```
23,01
45,02
67,04
-1
```

创建的链表：

```
23,01
45,02
67,04
```

请输入要插入结点的成绩和ID：

```
78,02
```

请输入要插入结点的位置,在这个位置后面插入新结点：

```
7
```

插入的节点不存在！插入后的链表：

```
23,01
45,02
67,04
```

输入提示："输入学生成绩和ID：\n"

输入格式： "%d,%s"

输出提示："创建的链表：\n"

输出格式："%d,%s\n"

输入提示："请输入要插入结点的成绩和ID:\n"

输入格式："%d,%s"

输入提示："请输入要插入结点的位置,在这个位置后面插入新结点：\n"

输入格式：%d

(如果输入的要插入位置不存在则输出提示： "插入的节点不存在！")

输出提示："插入后的链表：\n"

输出格式："%d,%s\n"

```
#include <stdio.h>
#include <stdlib.h>
#include<string.h>
struct grade {
    int score;
    char ID[6];
    struct grade *next;
};
typedef struct grade NODE;
struct grade *create();
NODE *insert(NODE *head, NODE *pnew, int i);
void display(NODE *head);
void Pfree(NODE *head);
int main() {
    struct grade *head, *pnew;//1
    int n;
    head = create();//1
    if (head == NULL)
        return 0;
    printf("创建的链表：\n");
    display(head); //1
    pnew=(NODE *)malloc(sizeof(NODE));
    printf("请输入要插入结点的成绩和ID:\n");
    scanf("%d,%s",&pnew->score,pnew->ID);//1
    getchar();
    printf("请输入要插入结点的位置,在这个位置后面插入新结点：\n");
    scanf("%d",&n); //1
    head=insert(head, pnew, n);//1
    printf("插入后的链表：\n");
    display(head);//1
    Pfree(head);//1
    return 0;
}
struct grade *create()  //2
{
    NODE *head, *tail, *pnew;//1
    int score;
    char ID[6];
    head = (NODE *)malloc(sizeof(NODE));
    if (head == NULL) {
        printf("创建失败！");
        return NULL;
    }
    head->next = NULL;
    tail = head;//1
    printf("输入学生成绩和ID：\n");
    while (1)  //1
    {
        scanf("%d,%s",&score,ID);
        if (score<0)//1
            break;
        pnew = (NODE *)malloc(sizeof(NODE));//1
        if (pnew == NULL) {
            printf("创建失败！");
            return NULL;
        }
        pnew->score = score;
        strcpy(pnew->ID, ID);
        pnew->next = NULL;
        tail->next = pnew;//1
        tail = pnew;//1
    }

    return head;//1
}
NODE *insert(NODE *head, NODE *pnew, int i) //2
{
    NODE *p; //1
    int j;

    p = head;
```

```
    for (j = 0; j<i&&p != NULL; j++)//1
        p = p->next;//1

    if (p == NULL)//1
    {
        printf("插入的节点不存在！");
        return head; //1
    }

    pnew->next = p->next;//1
    p->next = pnew; //1
    return head; //1
}

void display(NODE *head) //2
{
    NODE *p; //1
    for (p = head->next; p != NULL; p = p->next)//2
        printf("%d,%s\n", p->score, p->ID);//1
}

void Pfree(NODE *head)  //2
{
    NODE *p, *q;//1
    p = head;
    while (p->next != NULL)//1
    {
        q = p->next;
        p->next = q->next;//1
        free(q); //1
    }
    free(p);
}
```

5.（20分）输入一个正整数n，然后输入n个整数，存于n元一维动态数组（该数组由动态存储分配函数创建）。

然后统计n个数组元素中正数的个数存于整型变量mp以及负数的个数存于整型变量mn。
要求定义如下格式函数实现统计。

```
void cnt(int *a, int n, int *p, int *q)
```

函数调用时，a的实参为动态一维数组首地址，p，q的实参分别为mp和mn的地址。
主函数中实现数据输入、cnt函数调用以及数据输出。
输入提示："Input n=\n"以及"Input %d integer numbers:\n"
输入n以及n个整数的格式为"%d"
输出格式："mp=%d,mn=%d\n"
若输入n值小于1，则不输出任何内容直接结束main函数。
输入输出示例：

```
Input n=
6
Input 6 integer numbers:
-5 -10 -3 0 -2 0
mp=0,mn=4
```

```
#include "stdio.h"
#include "stdlib.h"
void cnt(int *a,int n,int *p,int *q)
{ int i;
  *p=*q=0;
  for(i=0;i<n;i++)
    { if(a[i]>0) (*p)++;
      if(a[i]<0) (*q)++;
    }
}
int main()
{  int *a,n,i,mp,mn;
   printf("Input n=\n");scanf("%d",&n);
   if(n<=0) return 0;
   a=(int *)malloc(sizeof(int)*n);
```

```
    if(!a) return 0;
    printf("Input %d integer numbers:\n",n);
    for(i=0;i<n;i++) scanf("%d",a+i);
    cnt(a,n,&mp,&mn);
    printf("mp=%d,mn=%d\n",mp,mn);
    free((void*)a);
    return 0;
}
```

@2016 HIT SSE